Discrete Event Simulation Analysis of a Web Application

CS 681: Assignment 2

Sameer Arvind Patil(23b1035) Harsh Suthar(23b1067)

March 2026

Outline

1. System Overview and Code overview
2. Setup 1: Simulated vs MVA vs Assignment-1
3. Setup 2: Confidence Intervals on General Webserver
4. Setup 3: Scheduling Policy Comparison (RR vs SJF)
5. Additional Analysis and Future Work

System Overview

- Multi-core web server simulator written in C++
- Models realistic web application performance characteristics
- Closed-loop user model: request $\rightarrow$ wait $\rightarrow$ think $\rightarrow$ repeat
- Thread-per-task model with configurable thread pool
- Supports multiple scheduling policies: Round-Robin (RR) and Shortest Job First (SJF)
- Discrete event simulation with warmup and transient removal
- Statistical analysis with confidence intervals

Code Overview: Key Files

- simulator.h – main event loop, simulation lifecycle, RNG seed handling, warmup/steady-state management
- event.h – event data structure, event types, priority ordering for the event queue
- core.h – per-core scheduling, run-queue, dispatching to cores, scheduling policy implementation
- threadworker.h – thread abstraction, context-switch handling, thread pool bookkeeping
- queueing_system.h – admission control, waiting queues, request enqueue/dequeue logic and drop policies
- request.h – request metadata (ids, service requirement, timeout, retries), completion and timeout flags
- common.h, main.cpp – shared types, configuration parsing, logging and metrics

For detailed documentation, refer the README.md in the simulator folder in the github repo

Key Performance Metrics

- **Average Response Time**: Mean time from request arrival to completion (Timed out requests not considered)
- **Throughput**: Total requests completed per second (goodput + badput)
- **Goodput**: Requests completed before timeout per second
- **Badput**: Timed-out requests completed per second
- **Timeout Rate**: Number of requests exceeding timeout threshold
- **Core Utilization**: Fraction of time cores are busy
- **Request Drop Rate**: Requests rejected due to full queue

Setup 1: Objective

Question: How well does the simulation match the Mean Value Analysis (MVA) theoretical model and Assignment 1 measurements?

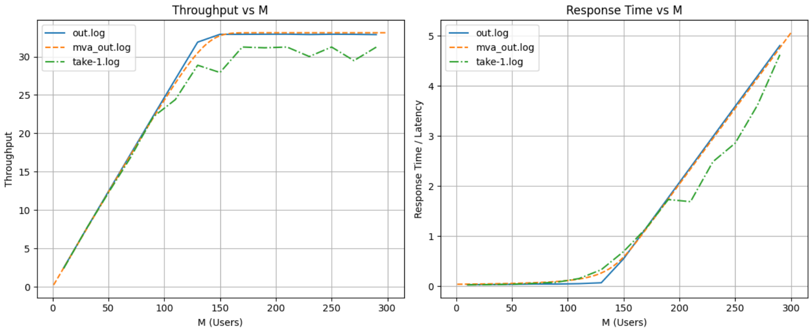
This setup validates the simulator by comparing against:

- MVA theoretical predictions
- Empirical measurements from Assignment 1
- Single simulator run with Assignment 1 parameters

Setup 1: Parameters

Parameter	Value
Number of Users	10 to 295 (step 20)
Service Time Distribution	Constant 30 ms
Number of Cores	1
Max Threads	345
Quantum (time slice)	10 ms
Context Switch Overhead	0.0 ms
Think Time (base)	4000 ms
Think Time (exponential mean)	10 ms
Timeout Range	30000–30010 ms
Simulation Time	180 seconds (warmup: 20 sec)

Setup 1: Comparison Results



The plot compares:

- **MVA**: Theoretical prediction line : mva_out.log
- **Assignment 1**: Measured data from real system : take-1.log
- **Simulation**: Discrete event simulation results : out.log

Analysis

What the plot shows

- Throughput increases nearly linearly for small values of M.
- After about 150 users, throughput starts to saturate, indicating system capacity limits.
- Response time remains low initially, but rises sharply as the load increases.
- The simulation output (`out.log`) is very close to the theoretical MVA prediction (`mva.out.log`).
- The real measurement data (`take-1.log`) follows the same trend, but shows slightly more variation due to real-system effects.

Conclusion

The three curves validate the model: the simulator and MVA match closely, and the measured system exhibits the expected performance trend.

Setup 2: Objective

Questions:

- How do performance metrics scale with increasing user load?
- What are the confidence intervals for response time?
- Where does the system saturate?

This setup performs 40 independent runs with varying user populations to compute statistically sound confidence intervals.

Setup 2: Parameters

Parameter	Value
Number of Users	40 to 3000 (step 40)
Service Time Distribution	Exponential, mean 60 ms
Number of Cores	4
Max Threads	256
Quantum (time slice)	10 ms
Context Switch Overhead	0.2 ms
Think Time (base)	3000 ms
Think Time (exponential mean)	200 ms
Timeout Range	20000–30000 ms
Simulation Time	180 seconds (warmup: 20 sec)
Independent Runs	40
Scheduling Policy	Round-Robin

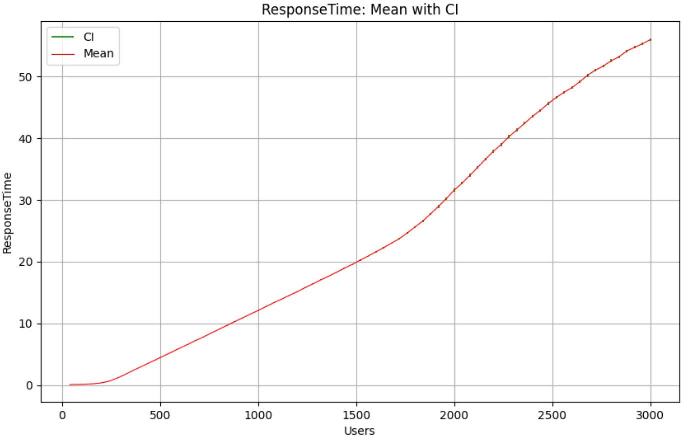
Setup 2: Statistical Methodology

- **Independent Replications:** 40 runs with different RNG seeds
- **Point Estimate:** Mean of the metric across all 40 runs for each user count
- **95% Confidence Interval:**

$$CI = \bar{X} \pm t_{0.025,39} \cdot \frac{S}{\sqrt{40}}$$

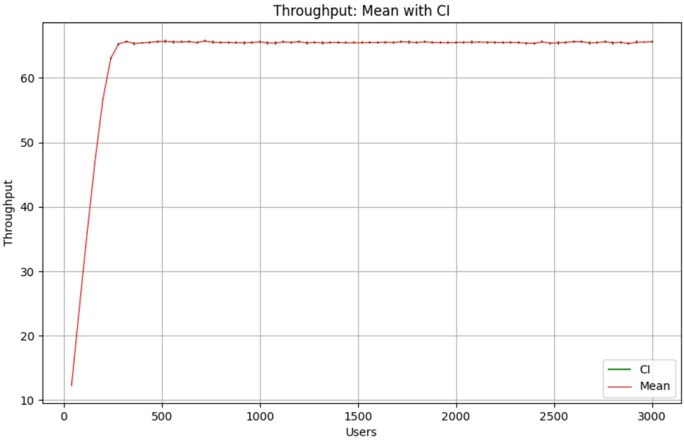
  where S is the sample standard deviation across runs, and $t_{0.025,39} \approx 2.02$ (Student's t-distribution)
- **Warmup Removal:** First 20 seconds discarded (empirically chosen)
- **Transient Analysis:** Statistics collected only after warmup (steady-state period)

Setup 2: Response Time Results



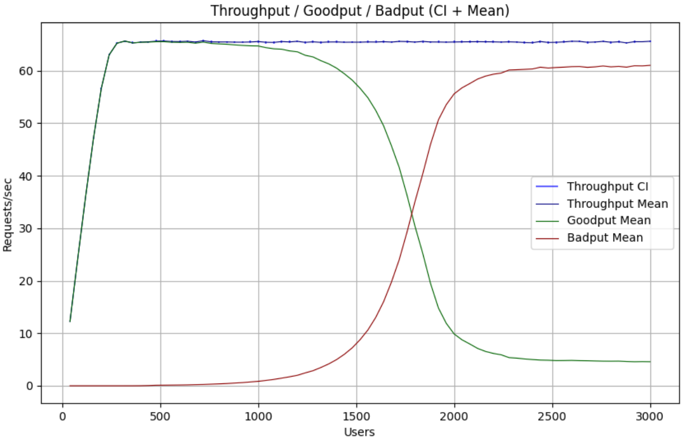
- Response time increases sub-linearly up to 500 users
- Increase in response time and confidence interval at high loads
- Saturation point near 220 users

Setup 2: Throughput Results



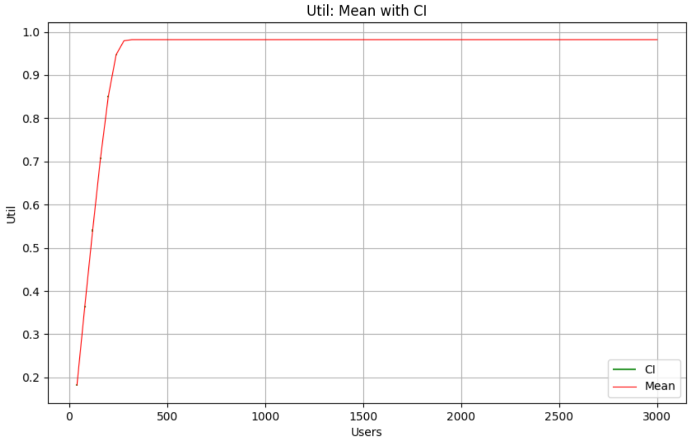
- Throughput increases with users up to saturation point
- Peak throughput $\approx$ 66 requests/sec at moderate load
- Plateaus after saturation due to server saturation

Setup 2: Goodput vs Badput



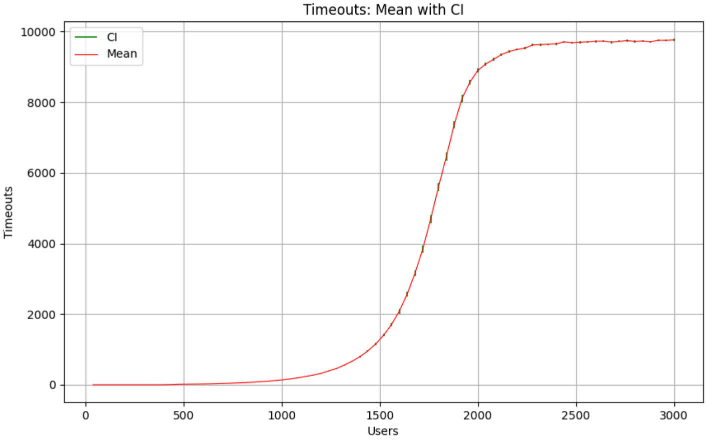
- Goodput: left to be done
- Badput: Timed-out requests that still completed
- Badput grows significantly with load (requests queue too long)

Setup 2: Utilization Results



- Core utilization increases with user load untill saturation
- Reaches $\approx 95\%$ at saturation (Did not achieve 100% utilization because of resources used in context-switches)

Setup 2: Timeout Analysis



- Few timeouts at low load left to be done
- Exponential growth in timeouts above 1000 users
- System SLA violation indicates saturation

Setup 2: Key Insights

- **Saturation Point**: System becomes resource-constrained around 250 users
- **Queueing Effects**: Visible degradation in response time and goodput
- **Thread Pool Limit**: 256 threads insufficient for extreme loads
- **Variability**: Confidence intervals widen significantly at high loads
- **Confidence**: 95% CI indicates statistical soundness lefttt

Setup 3: Objective

Questions:

- How much does scheduling policy impact performance?
- Is SJF better than RR for web server workloads (in terms of response time)?

This setup compares two scheduling policies:

- **Round-Robin (RR)**: Fair, simple scheduling
- **Shortest Job First (SJF)**: Prioritizes shorter jobs

Setup 3: Parameters

Parameter	RR	SJF
Number of Users	5–3000 (step 10)	5–3000 (step 10)
Service Time	Exp(60) ms	Exp(60) ms
Number of Cores	4	4
Max Threads	256	256
Quantum	10 ms	100 ms
Context Switch Overhead	0.2 ms	1.0 ms
Think Time (base)	3000 ms	3000 ms
Mean Think Time (exponential)	200 ms	200 ms
Timeout Range	20000–30000 ms	20000–30000 ms

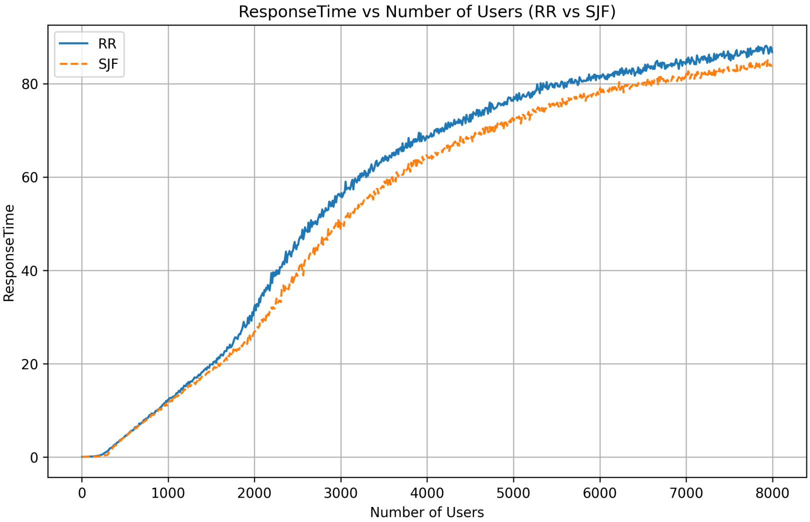
Real think time is sum of base + exponential component.

Setup 3: Rationale

- **Different Service Time**: Creates variability for SJF to optimize
- **Multiple Cores**: Real-world scenario with parallelism
- **Wide User Range**: Captures behavior from light to extreme load

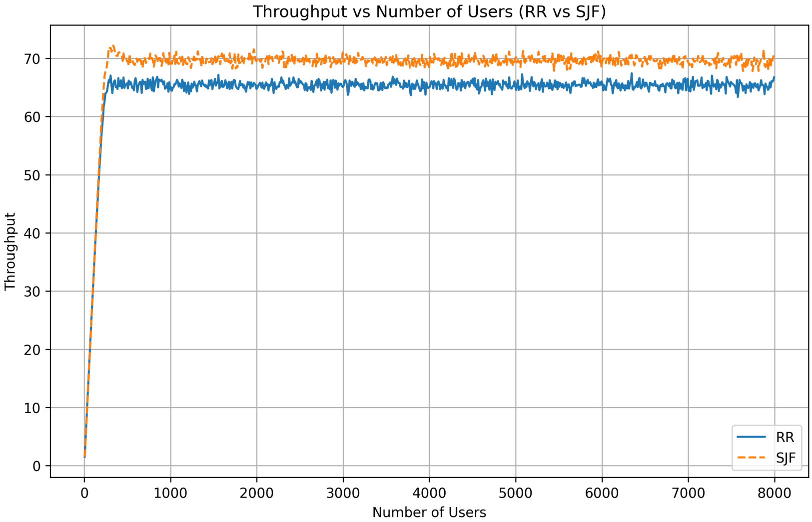
We expect SJF to outperform RR in terms of response time. leftt

Setup 3: RR vs SJF Response Time



- SJF generally shows lower response time

Setup 3: RR vs SJF Throughput



- Higher throughput for SJF

Setup 3: Analysis

- **SJF Advantage**: Better response time for short jobs
- **Saturation Behavior**: Both policies degrade similarly at extreme load
- **Recommendation**:
  - Use SJF for systems with highly variable service times
  - RR is simpler and adequate for balanced workloads
  - Consider hybrid approaches for critical services

Conclusion

- **Simulator Validation**: Discrete event simulation closely matches MVA theory and experimental data
- **Performance Characterization**: Comprehensive metrics with confidence intervals
- **Scheduling Impact**: SJF outperforms RR for variable workloads (in term of response time and throughput)

Code Structure

Core Components:

- `simulator.h/cpp`: Main event loop lefttt
- `request.h`: Request tracking
- `core.h/cpp`: Per-core scheduling
- `threadworker.h`: Thread abstraction

Analysis:

- `plot.py`: Basic metrics plotting
- `confidence_plotter.py`: CI calculations
- `mix_plot.py`: Multi-file plotter
- Scripts: Runner shells for all setups